

dlt (data load tool)

ingestão de dados do jeito Pythônico

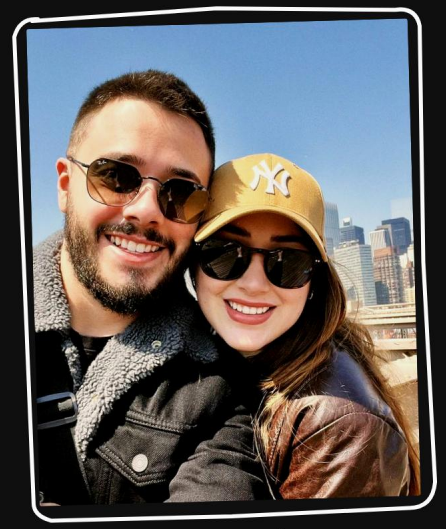
Alan Molina · Engenheiro de Dados

alanmmolina.com

whoami


2015
2019
2020
2021
2024
hoje
UNESP
Engenharia de Controle e Automação
Visagio
Consultor
Engenharia de Dados
iFood
Arquiteto de Dados
Engenheiro de Software
Branching Minds
Data Engineer

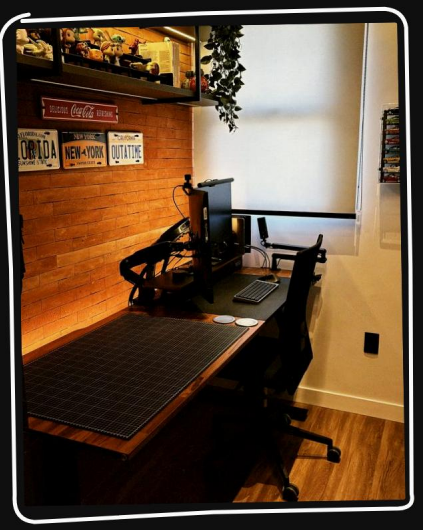
whoami



Ingrid



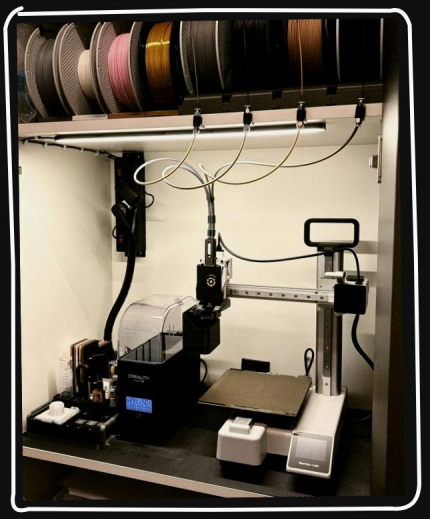
Maggie



escritório

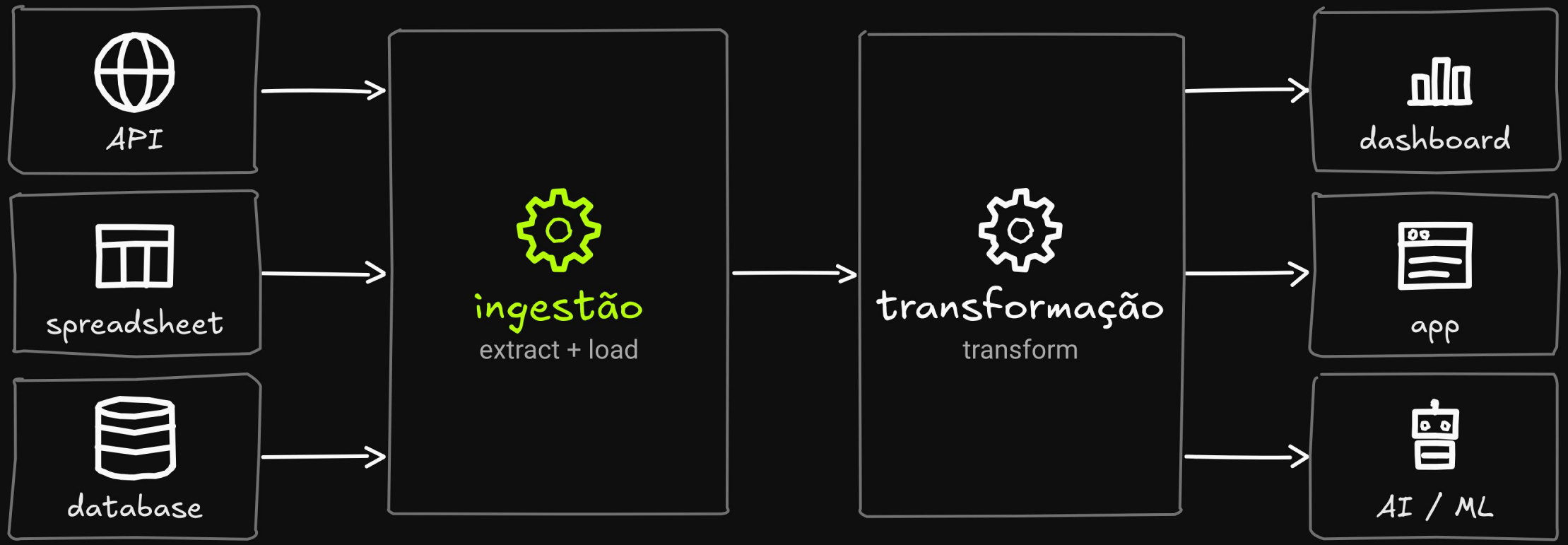


oficina

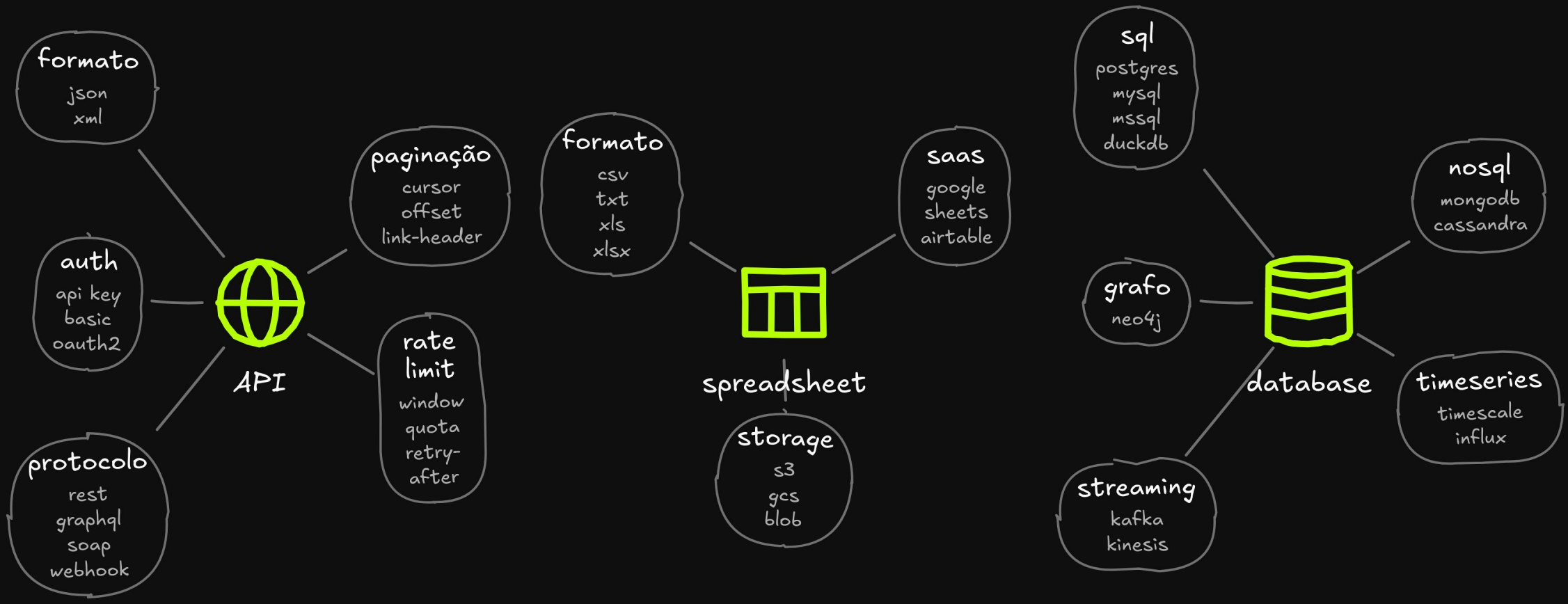


impressão 3D

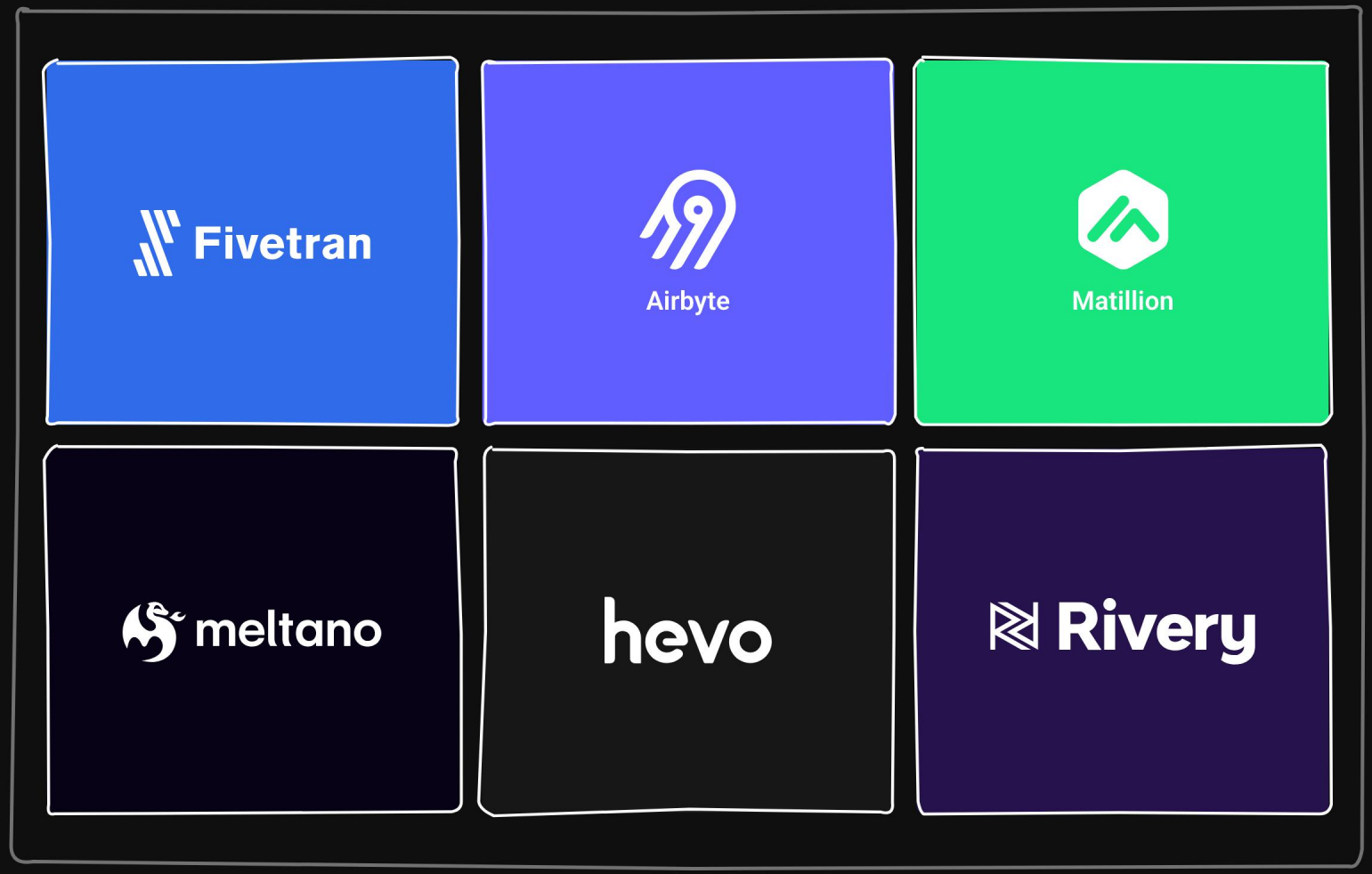
Pipeline de Dados



Nenhuma fonte é igual



Plataformas de Conectores



maquinário à parte

componente separado na arquitetura, com ciclo de vida e overhead operacional próprios

fonte particular

quando a fonte foge do padrão, esbarra nas restrições justamente onde você precisa de flexibilidade

Scripts caseiros

ingestor.py

```python
import json
import requests

URL = "https://dummyjson.com/products"
items, skip = [], 0
while True:
    params = {"limit": 10, "skip": skip}
    response = requests.get(URL, params)
    batch = response.json()
    items += batch["products"]
    if len(batch["products"]) < 10:
        break
    skip += 10

with open("items.json", "w") as file:
    json.dump(items, file)
```

síncrono · um request por vez

client.py

```python
import requests

def page(url,skip):
    params = {"limit": 10, "skip": skip}
    response = requests.get(url, params)
    return response.json()["products"]


class Store:
    def __init__(self, url):
        self.url = url

    def items(self):
        skip = 0
        while True:
            batch = page(self.url, skip)
            yield from batch
            if len(batch) < 10:
                break
            skip += 10
```

classe · devolve um gerador

fetcher.py

```python
import asyncio
import httpx

async def fetch(client,url):
    response = await client.get(url)
    return response.json()

async def all_pages(client, urls):
    jobs = []
    for url in urls:
        jobs.append(fetch(client, url))
    return await asyncio.gather(*jobs)
```

assíncrono · páginas em paralelo

$ pip install dlt

vive na sua codebase

roda onde Python rodar, de forma integrada ao codebase

customizável e extensível

a linguagem de extensão é o próprio Python da codebase

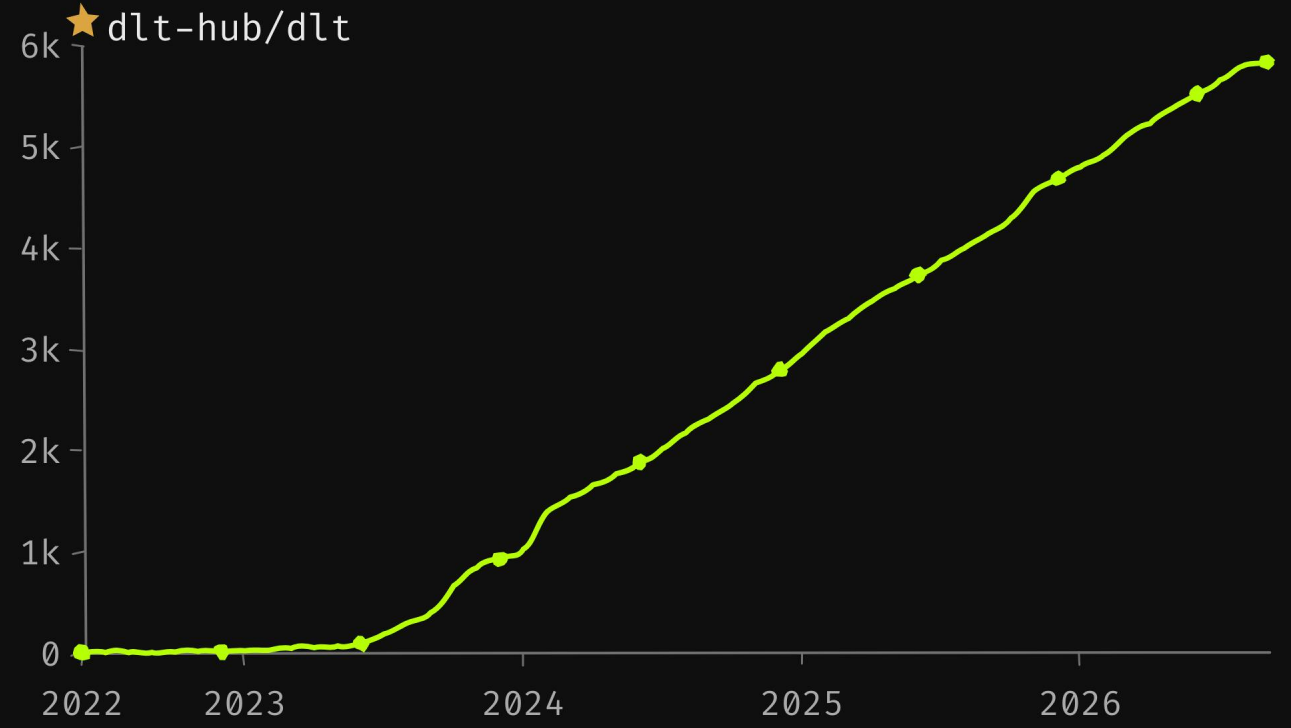
amigável a coding agents

arquitetura declarativa que agentes conseguem utilizar e aplicar facilmente

5,8k
stars no GitHub

5,7M
downloads/mês no PyPI

10k+
empresas em produção



dltHub

- open source · licença Apache 2.0
- a empresa · duas camadas adicionais

Agentic Layer

AI Harness

a interface entre o coding agent e o dltHub. Skills, comandos, regras e MCP que mantêm o agente nas melhores práticas

Context Graph

a memória de longo prazo do agente: traces, logs e schema de cada pipeline. Se algo quebra, ele sabe o que mudou e onde

Managed Infrastructure Layer

execução + orquestração

o cliente é responsável somente por escrever o código. O dltHub agenda, executa e gerencia a infraestrutura

observabilidade

logs, métricas, alertas de cada execução já embutidos na infraestrutura gerenciada pelo dltHub

Conceitos fundamentais



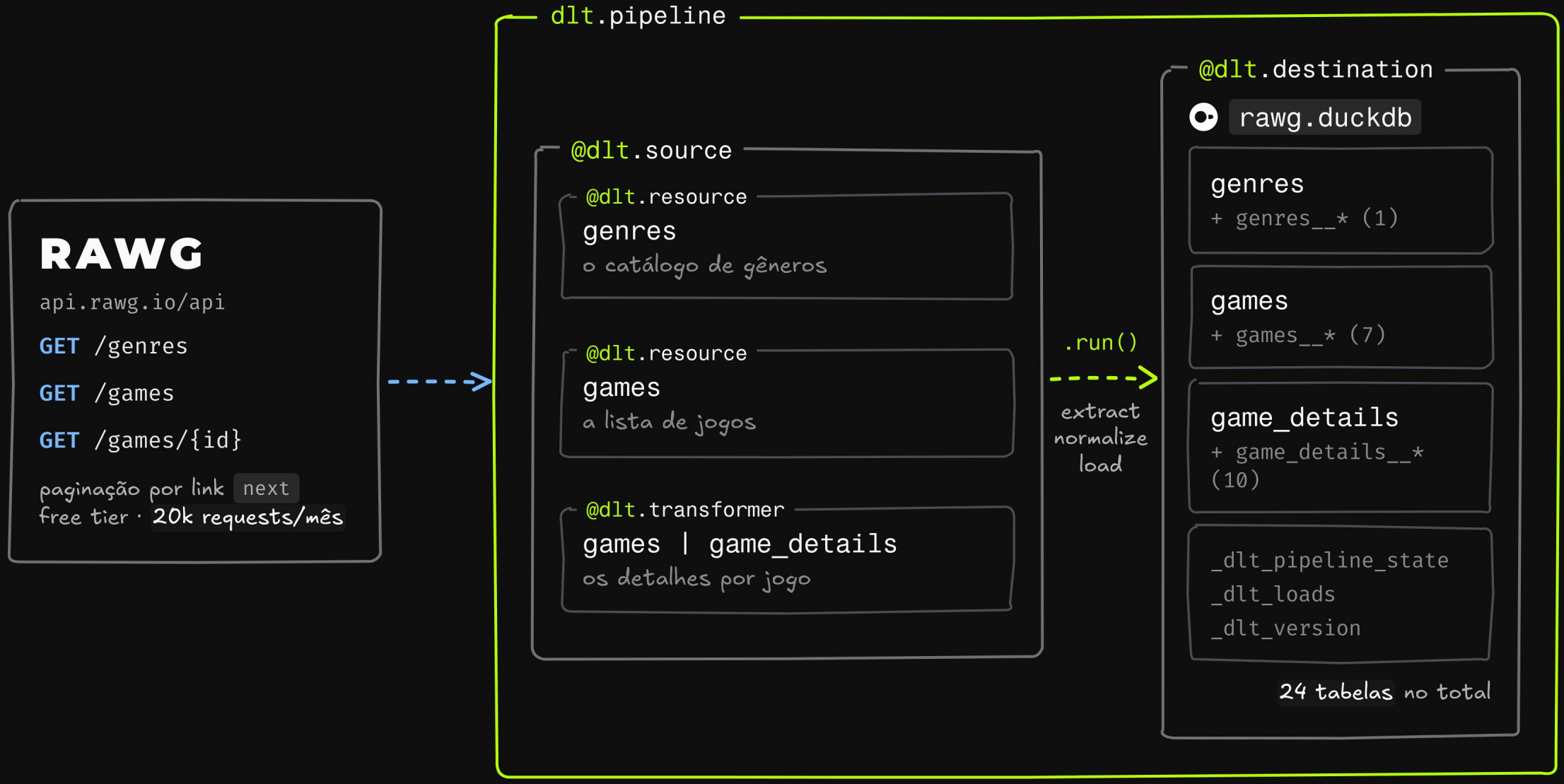
Três fases distintas



por que isso importa

`extract` é a única fase que conversa com a fonte; `load`, a única com o destino. Entre elas, tudo é persistido em disco: fonte lenta não trava o destino, falha de rede no meio do load não perde o que já foi extraído, cada fase pode ser reprocessada sozinha.

Nosso objeto de estudo



@dlt.source

uma função Python que retorna um ou mais `resources`

```python
import dlt
from dlt.sources.helpers.rest_client import RESTClient
from dlt.sources.helpers.rest_client.auth import APIKeyAuth
from dlt.sources.helpers.rest_client.paginators import JSONLinkPaginator

@dlt.source(name="rawg")
def rawg(api_key: str = dlt.secrets.value) -> list[dlt.sources.DltResource]:
    client = RESTClient(
        base_url="https://api.rawg.io/api",
        auth=APIKeyAuth(name="key", location="query", api_key=api_key),
        paginator=JSONLinkPaginator(next_url_path="next"),
    )
    return [
        genres(client=client),
        (games_resource := games(client=client)),
        games_resource | game_details(client=client),
    ]
```

.dlt/secrets.toml

```toml
[sources.rawg]
api_key = "RAWG_API_KEY"
```

a key é injetada como query param no request

```
GET /api/games?key=RAWG_API_KEY
```

a próxima página é encontrada através do link de retorno no campo next

```
{
  "next": "…/api/games?key=…&page=2",
  "count": 500518,
  "results": [ { "id": 51325, … }, … ]
}
```

rawg.io

@dlt.resource

uma função Python que "yielda" dados

dict, list, DataFrames Pandas/Polars, tabelas Arrow, modelos Pydantic

estratégia de escrita

nome da tabela

lazy
item a item

```python
...

@dlt.resource(
    write_disposition="replace",
)
def genres(
    client: RESTClient,
) -> Iterator[dict[str, Any]]:
    for page in client.paginate(
        path="genres",
        params={"page_size": 40},
        data_selector="results",
    ):
        yield from page
```


```
{
    "count": 19,
    "next": null,
    "results": [
      {
        "id": 4,
        "name": "Action",
        "slug": "action",
        "games_count": 192215,
        "games": [
          { "id": 51325, "name": "The Last of Us Part II",
            "added": 7504 },
          { "name": "Red Dead Redemption 2", … },
          …
        ]
      },
      {
        "id": 3,
        "name": "Adventure",
        "games_count": 152521,
        "games": [
          { "name": "Ghost of Tsushima", … },
          { "name": "Horizon Forbidden West", … },
          …
        ]
      },
      …
    ]
  }
```


genres

genres__games

rawg.io

@dlt.resource

respostas de APIs geralmente utilizam listas e estruturas aninhadas

upsert
chave do upsert

cursor
incremental

mais novos
primeiro

```python
...

@dlt.resource(
    write_disposition="merge",
    primary_key="id",
)
def games(
    client: RESTClient,
    updated_at=dlt.sources.incremental(
        cursor_path="updated",
        initial_value="1970-01-01T00:00:00",
        row_order="desc",
    ),
) → Iterator[dict[str, Any]]:
    for page in client.paginate(
        path="games",
        params={"ordering": "-updated"},
        data_selector="results",
    ):
        yield from page
```

```
{
    "next": "…page=2",
    "results": [
      {
        "id": 51325,
        "slug": "the-last-of-us-part-2",
        "name": "The Last of Us Part II",
        "released": "2020-06-19",
        "rating": 4.41,
        "ratings_count": 2053,
        "metacritic": 93,
        "genres": [
          { "name": "Action", … },
          { "name": "Adventure", … },
          …
        ],
        "tags": [
          { "name": "Singleplayer", … },
          { "name": "Atmospheric", … },
          …
        ],
        "platforms": [
          { "platform": { "name": "PlayStation 4", … }, … },
          { "platform": { "name": "PlayStation 5", … }, … }
        ],
        "stores": [
          { "store": { "name": "PlayStation Store", … }, … }
        ],
        "esrb_rating": { "name": "Mature", … },
        …
      }
    ]
  }
```

games

games__genres

games__tags

games__platforms

games__stores

esrb_rating__name

rawg.io

@dlt.transformer

quando precisamos de dados para extrair mais dados

`resources` são encadeados sem duplicação de código

chamadas em paralelo

objeto único

```python
...

@dlt.transformer(
    write_disposition="merge",
    primary_key="id",
    parallelized=True,
)
def game_details(
    game_item: dict[str, Any],
    client: RESTClient,
) → dict[str, Any]:
    return client.get(
        path=f"games/{game_item['id']}",
    ).json()
```

resource games busca os itens

```
{ "id": 51325,
    "name": "The Last of Us Part II",
    "rating": 4.41, … }
```

id 51325

os ids são utilizados para fazer o request por game

```
GET /api/games/51325?key=RAWG_API_KEY
```

traz detalhes adicionais para cada game

```
{ "id": 51325,
    "description": "…",
    "website": "…",
    "alternative_names": [ "TLOU2", … ],
    "metacritic_platforms": [
      { "platform": {
          "name": "PlayStation 4" },
        … } ],
    … }
```

rawg.io

dlt.Schema

a forma

a política

```python
@dlt.resource(
    write_disposition="merge",
    columns=Game,
    primary_key="id",
    schema_contract={"columns": "freeze"},
)
def games(client: RESTClient,…):
    ...
```

columns

dict

```
{
  "id": {"data_type": "bigint"}
}
```

list

```
[{
  "name": "id",
  "data_type": "bigint"
}]
```

Pydantic

```python
class Game(BaseModel):
  id: int
```

pyarrow

```python
pa.schema([
  ("id", pa.int64())
])
```

schema_contract

evolve

schema muda
dados são carregados

freeze

schema não muda
dados não são carregados

discard_row

schema não muda
linha não é carregada

discard_value

schema não muda
valor não é carregado

@dlt.destination

trocar de destino é mudar uma string

utilizar as definições padrão do destino

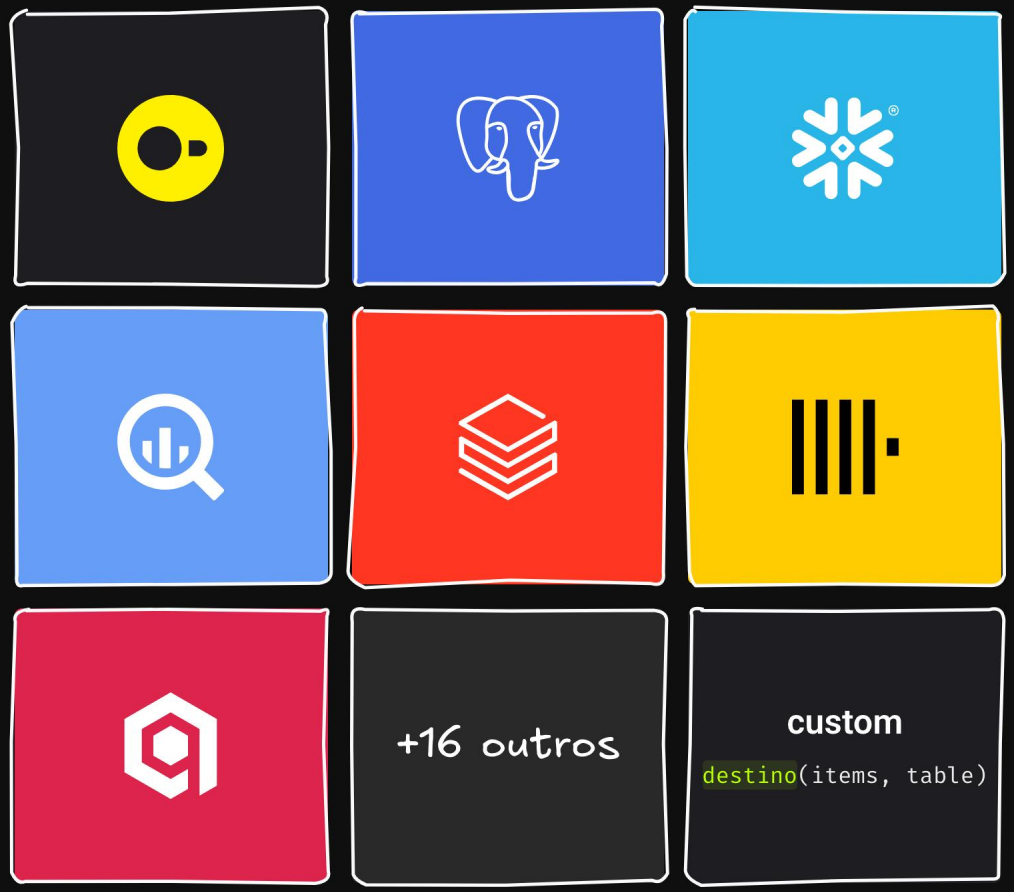
```
pipeline = dlt.pipeline("rawg", destination="duckdb")
```

configurar o destino com os argumentos da factory

```
datalake = filesystem(
      "s3://dlt-datalake",
      destination_name="datalake",
      naming_convention="sql_cs_v1",
      layout="raw/{table_name}/{load_id}.{ext}",
  )
```

criar um destino customizado com uma função

```
@dlt.destination
def destino(items, table):
    ...
```

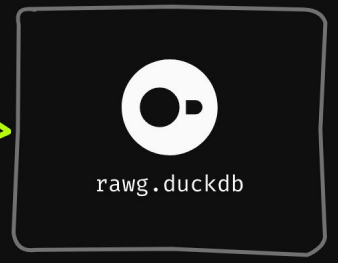


dlt.pipeline

RAWG
api.rawg.io/api

```
pipeline = dlt.pipeline("rawg", destination="duckdb")
info = pipeline.run(rawg_source())

>>> info.load_packages[0].load_id
'1790731675.145563'
```



~/.dlt/pipelines/rawg/
normalize/extracted/1790731675.145563/new_jobs/
games.014597893b.0.typed-jsonl.gz

```
{ "id": 3498,
    "slug": "grand-theft-auto-v",
    "name": "Grand Theft Auto V",
    "released": "2013-09-17",
    "tba": false,
    "background_image": "https://…jpg",
    "rating": 4.47,
    "rating_top": 5, … }
```

extract

~/.dlt/pipelines/rawg/
schemas/
rawg.schema.json

```
"name": "rawg",
  "tables": { "games": {
    "columns": {
      "id": { "data_type": "bigint",
              "primary_key": true },
      "name": { "data_type": "text" },
      "rating": { "data_type": "double" }
      … } }
```

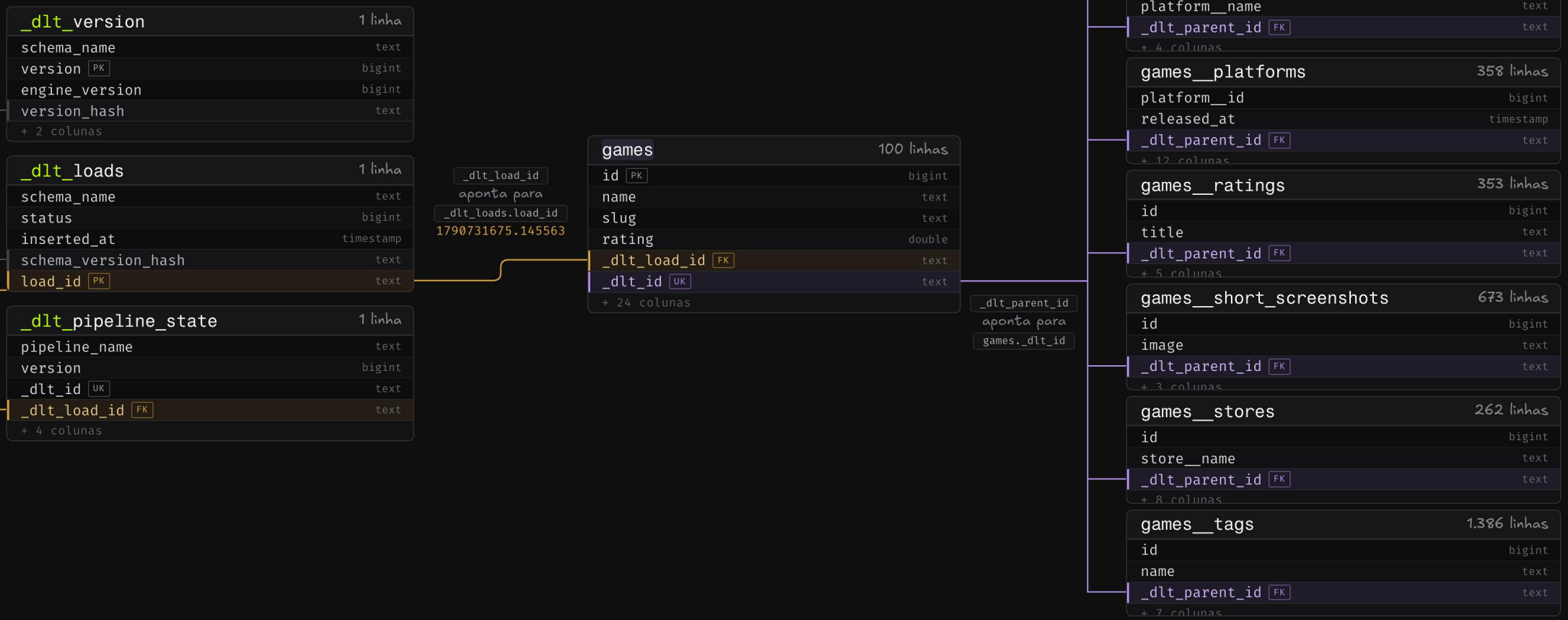
normalize

~/.dlt/pipelines/rawg/
load/loaded/1790731675.145563/completed_jobs/
games.5c5a686297.0.insert_values.gz

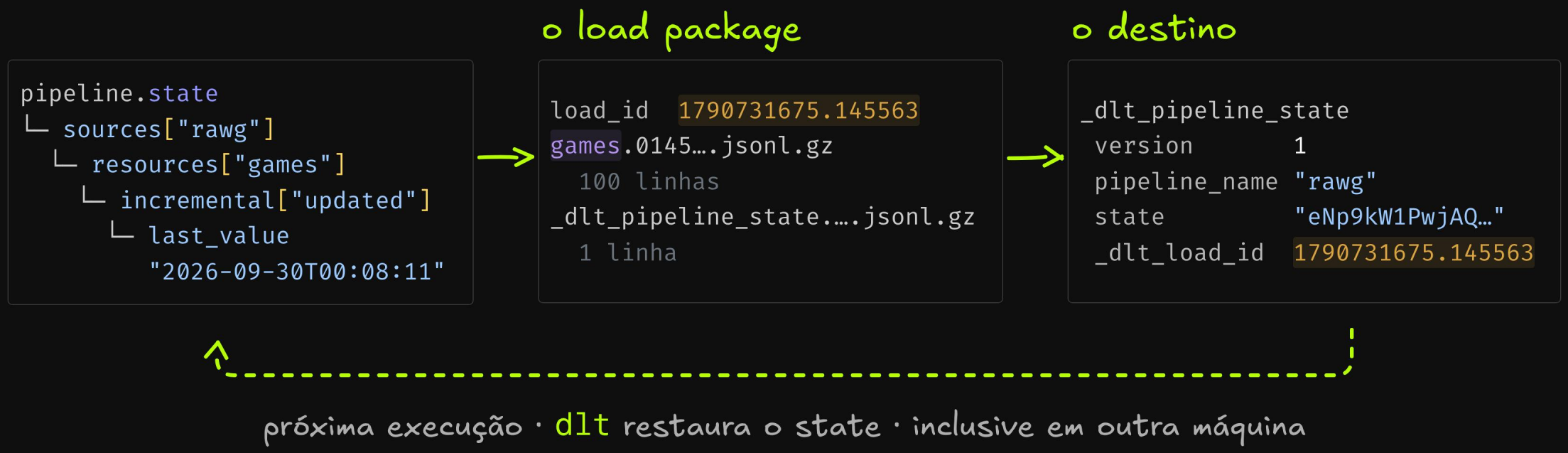
```
INSERT INTO {}("id","slug","name",
    "released","tba","background_image",
    "rating","rating_top", …)
  VALUES
  (3498, E'grand-theft-auto-v',
   E'Grand Theft Auto V',
   E'2013-09-17', False,
   E'https://…jpg', 4.47, 5, …)
```

load

rawg.duckdb


_dlt_version
1 linha
schema_name text
version PK bigint
engine_version bigint
version_hash text
+ 2 colunas
_dlt_loads
1 linha
schema_name text
status bigint
inserted_at timestamp
schema_version_hash text
load_id PK text
_dlt_pipeline_state
1 linha
pipeline_name text
version bigint
_dlt_id UK text
_dlt_load_id FK text
+ 4 colunas
_dlt_load_id
aponta para
_dlt_loads.load_id
1790731675.145563
games
100 linhas
id PK bigint
name text
slug text
rating double
_dlt_load_id FK text
_dlt_id UK text
+ 24 colunas
_dlt_parent_id
aponta para
games._dlt_id
games__genres
225 linhas
id bigint
name text
_dlt_parent_id FK text
+ 6 colunas
games__parent_platforms
320 linhas
platform__id bigint
platform__name text
_dlt_parent_id FK text
+ 4 colunas
games__platforms
358 linhas
platform__id bigint
released_at timestamp
_dlt_parent_id FK text
+ 12 colunas
games__ratings
353 linhas
id bigint
title text
_dlt_parent_id FK text
+ 5 colunas
games__short_screenshots
673 linhas
id bigint
image text
_dlt_parent_id FK text
+ 3 colunas
games__stores
262 linhas
id bigint
store__name text
_dlt_parent_id FK text
+ 8 colunas
games__tags
1.386 linhas
id bigint
name text
_dlt_parent_id FK text
+ 7 colunas

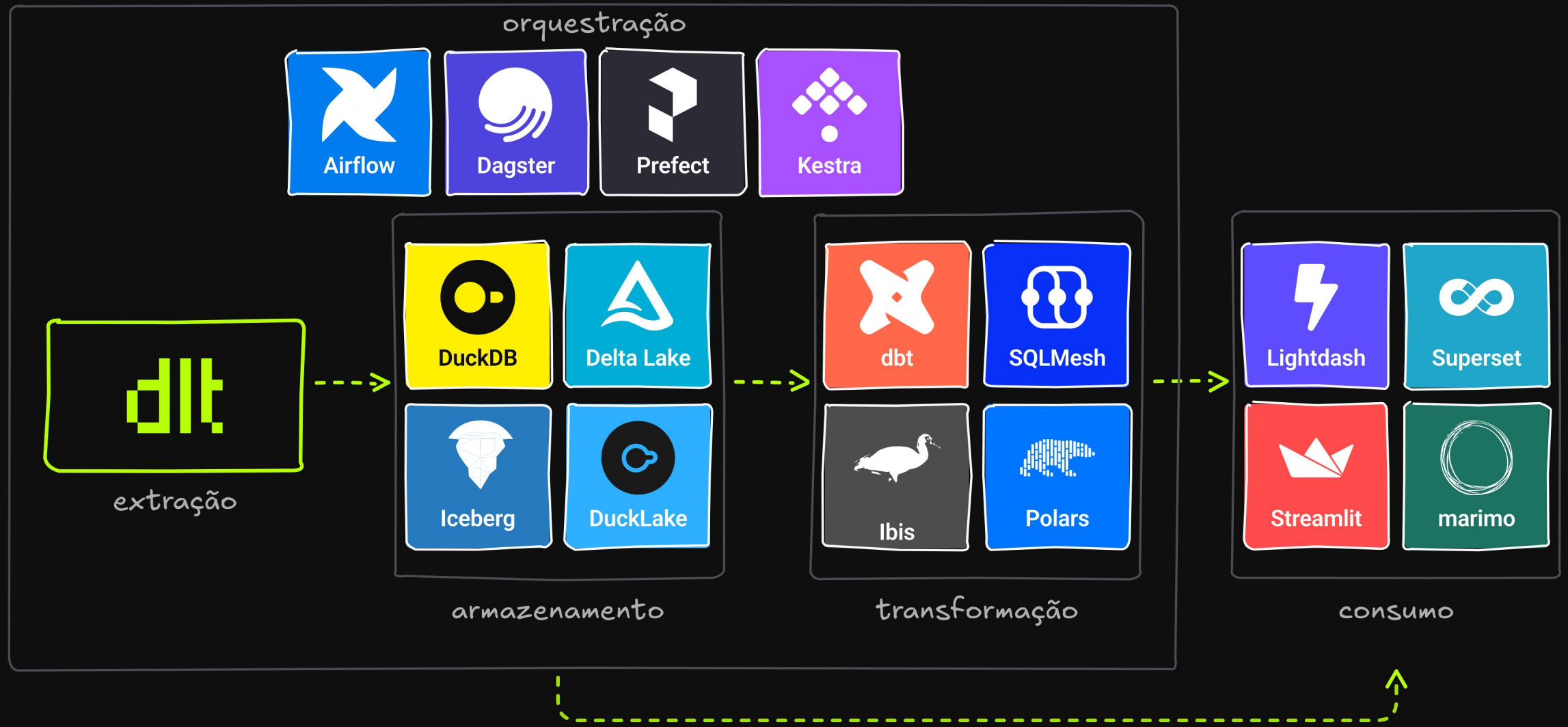
dlt.state



viaja junto, volta sozinho

o state é transportado dentro do load package, com o mesmo `load_id` e commit dos dados. Na próxima execução, o dlt restaura o state da última carga completa. Cursores incrementais são salvos automaticamente no state.

Plataforma de Dados Moderna





obrigado!

perguntas?

alanmmolina.com

contato@alanmmolina.com

in/alanmmolina

@alanmmolina